

Microsoft Fabric · WMS SQL-First Medallion

PART 2 — SILVER TRANSFORM

Notebook: nb_silver_transform_sql · 15 Delta Tables · 16 Cells · 0 Lines of PySpark

Philosophy

- Zero PySpark. Every transform is a SQL CTAS with CTE chains.
- Deduplication uses ROW_NUMBER() window function — no .dropDuplicates().
- Reference lookups joined inline from Bronze to avoid inter-Silver ordering dependencies.
- All derived columns (margins, rates, flags, buckets) are CASE WHEN or arithmetic inside SELECT.
- Run Bronze validation (Part 1, Cell 21) before this notebook.

Silver Table Transformations

CELL 1 — silver.silver_orders | PK: order_id

Source: bronze.raw_orders · Key derived cols: days_to_required, order_month, order_year

```
CREATE OR REPLACE TABLE silver.silver_orders
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY order_id ORDER BY _loaded_at DESC) AS rn
  FROM bronze.raw_orders WHERE order_id IS NOT NULL
)
SELECT
  order_id, customer_id, warehouse_id,
  TO_DATE(order_date) AS order_date,
  TO_DATE(required_date) AS required_date,
  CAST(order_value AS DOUBLE) AS order_value,
  CAST(total_lines AS INT) AS total_lines,
  UPPER(TRIM(status)) AS status,
  UPPER(TRIM(priority)) AS priority,
  channel,
  DATEDIFF(TO_DATE(required_date), TO_DATE(order_date)) AS days_to_required,
  DATE_FORMAT(TO_DATE(order_date), 'yyyy-MM') AS order_month,
  YEAR(TO_DATE(order_date)) AS order_year,
  current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;
```

CELL 2 — silver.silver_order_lines | PK: line_id

Source: bronze.raw_order_lines + bronze.raw_products (lookup) · Key derived cols: fulfillment_rate, line_margin, category/subcategory

```
CREATE OR REPLACE TABLE silver.silver_order_lines
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
  PARTITION BY line_id ORDER BY _loaded_at DESC) AS rn
  FROM bronze.raw_order_lines WHERE line_id IS NOT NULL),
typed AS (SELECT line_id, order_id, sku,
```

```

CAST(qty_ordered AS INT) AS qty_ordered,
CAST(qty_fulfilled AS INT) AS qty_fulfilled,
CAST(unit_price AS DOUBLE) AS unit_price,
CAST(line_value AS DOUBLE) AS line_value,
CASE WHEN CAST(qty_ordered AS INT) > 0
THEN CAST(qty_fulfilled AS DOUBLE) / CAST(qty_ordered AS DOUBLE)
ELSE 0.0 END AS fulfillment_rate
FROM deduped WHERE rn = 1),
with_product AS (SELECT t.*, p.unit_cost, p.category, p.subcategory,
t.line_value - (t.qty_fulfilled * p.unit_cost) AS line_margin
FROM typed t LEFT JOIN bronze.raw_products p ON t.sku = p.sku)
SELECT *, current_timestamp() AS _loaded_at FROM with_product;

```

CELL 3 — silver.silver_customers | PK: customer_id

Source: bronze.raw_customers · Key derived cols: segment/account_status normalized, customer_since_year

```

CREATE OR REPLACE TABLE silver.silver_customers
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY customer_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_customers WHERE customer_id IS NOT NULL)
SELECT
customer_id, first_name, last_name, email, phone,
UPPER(TRIM(segment)) AS segment,
UPPER(TRIM(account_status)) AS account_status,
CAST(credit_limit AS DOUBLE) AS credit_limit,
city, state, region, country,
TO_DATE(created_date) AS created_date,
YEAR(TO_DATE(created_date)) AS customer_since_year,
current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;

```

CELL 4 — silver.silver_products | PK: sku

Source: bronze.raw_products · Key derived cols: margin_pct, volume_cuft

```

CREATE OR REPLACE TABLE silver.silver_products
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY sku ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_products WHERE sku IS NOT NULL)
SELECT sku, product_name, category, subcategory, supplier_id,
CAST(unit_cost AS DOUBLE) AS unit_cost,
CAST(unit_price AS DOUBLE) AS unit_price,
CAST(weight_kg AS DOUBLE) AS weight_kg,
CAST(reorder_point AS INT) AS reorder_point,
CAST(lead_time_days AS INT) AS lead_time_days,
CAST(is_hazmat AS BOOLEAN) AS is_hazmat,
CAST(is_perishable AS BOOLEAN) AS is_perishable,
CASE WHEN CAST(unit_price AS DOUBLE) > 0
THEN (CAST(unit_price AS DOUBLE) - CAST(unit_cost AS DOUBLE))
/ CAST(unit_price AS DOUBLE)
ELSE 0.0 END AS margin_pct,
(CAST(length_cm AS DOUBLE) * CAST(width_cm AS DOUBLE)
* CAST(height_cm AS DOUBLE)) / 28316.8 AS volume_cuft,
current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;

```

CELL 5 — silver.silver_inventory | PK: inventory_id

Source: bronze.raw_inventory + bronze.raw_products (lookup) · Key derived cols: inventory_value, below_reorder flag, utilization_rate

```

CREATE OR REPLACE TABLE silver.silver_inventory
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY inventory_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_inventory WHERE inventory_id IS NOT NULL AND sku IS NOT NULL),
typed AS (SELECT inventory_id, sku, warehouse_id, location_id,
CAST(qty_on_hand AS INT) AS qty_on_hand,
CAST(qty_available AS INT) AS qty_available,
CAST(unit_cost AS DOUBLE) AS unit_cost,
CAST(reorder_point AS INT) AS reorder_point,
TO_TIMESTAMP(last_counted_at) AS last_counted_at
FROM deduped WHERE rn = 1),
enriched AS (SELECT t.*,
t.qty_on_hand * t.unit_cost AS inventory_value,
t.qty_available < t.reorder_point AS below_reorder,
CASE WHEN t.reorder_point > 0
THEN CAST(t.qty_on_hand AS DOUBLE) / t.reorder_point
ELSE NULL END AS utilization_rate,
p.product_name, p.category, p.subcategory
FROM typed t LEFT JOIN bronze.raw_products p ON t.sku = p.sku)
SELECT *, current_timestamp() AS _loaded_at FROM enriched;

```

CELL 6 — silver.silver_inventory_transactions | PK: txn_id

Source: bronze.raw_inventory_transactions · Key derived cols: txn_date, txn_month, qty_mismatch_flag

```

CREATE OR REPLACE TABLE silver.silver_inventory_transactions
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY txn_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_inventory_transactions WHERE txn_id IS NOT NULL)
SELECT txn_id, inventory_id, sku, warehouse_id, location_id,
txn_type, reference_id, reference_type,
CAST(qty_change AS INT) AS qty_change,
CAST(qty_before AS INT) AS qty_before,
CAST(qty_after AS INT) AS qty_after,
CASE WHEN CAST(qty_before AS INT) + CAST(qty_change AS INT)
<> CAST(qty_after AS INT)
THEN TRUE ELSE FALSE END AS qty_mismatch_flag,
TO_TIMESTAMP(txn_timestamp) AS txn_timestamp,
TO_DATE(txn_timestamp) AS txn_date,
DATE_FORMAT(TO_DATE(txn_timestamp), 'yyyy-MM') AS txn_month,
performed_by,
current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;

```

CELL 7 — silver.silver_shipments | PK: shipment_id

Source: bronze.raw_shipments + bronze.raw_carriers (lookup) · Key derived cols: delay_days, cost_per_lb, carrier attributes

```

CREATE OR REPLACE TABLE silver.silver_shipments
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY shipment_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_shipments WHERE shipment_id IS NOT NULL),
typed AS (SELECT shipment_id, order_id, carrier_id, origin_warehouse_id,
TO_DATE(ship_date) AS ship_date,
TO_DATE(estimated_delivery) AS estimated_delivery,
TO_DATE(actual_delivery) AS actual_delivery,
UPPER(TRIM(status)) AS status,
CAST(shipping_cost AS DOUBLE) AS shipping_cost,

```

```

CAST(weight_lbs AS DOUBLE) AS weight_lbs,
CAST(is_on_time AS BOOLEAN) AS is_on_time,
CASE WHEN actual_delivery IS NOT NULL AND estimated_delivery IS NOT NULL
THEN DATEDIFF(TO_DATE(actual_delivery), TO_DATE(estimated_delivery))
ELSE 0 END AS delay_days,
CASE WHEN CAST(weight_lbs AS DOUBLE) > 0
THEN CAST(shipping_cost AS DOUBLE) / CAST(weight_lbs AS DOUBLE)
ELSE NULL END AS cost_per_lb
FROM deduped WHERE rn = 1),
with_carrier AS (SELECT t.*, c.carrier_name, c.service_level,
c.avg_transit_days, c.cost_per_lb AS carrier_rate_per_lb
FROM typed t LEFT JOIN bronze.raw_carriers c ON t.carrier_id = c.carrier_id)
SELECT *, current_timestamp() AS _loaded_at FROM with_carrier;

```

CELL 8 — silver.silver_purchase_orders | PK: po_id

Source: *bronze.raw_purchase_orders + bronze.raw_suppliers (lookup)* · **Key derived cols:** *actual_lead_days, days_late*

```

CREATE OR REPLACE TABLE silver.silver_purchase_orders
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY po_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_purchase_orders WHERE po_id IS NOT NULL),
typed AS (SELECT po_id, supplier_id, sku, warehouse_id,
TO_DATE(po_date) AS po_date,
TO_DATE(expected_delivery) AS expected_delivery,
TO_DATE(actual_delivery) AS actual_delivery,
UPPER(TRIM(status)) AS status,
CAST(qty_ordered AS INT) AS qty_ordered,
CAST(unit_cost AS DOUBLE) AS unit_cost,
CAST(total_cost AS DOUBLE) AS total_cost,
CAST(is_on_time AS BOOLEAN) AS is_on_time,
CASE WHEN actual_delivery IS NOT NULL
THEN DATEDIFF(TO_DATE(actual_delivery), TO_DATE(po_date))
ELSE NULL END AS actual_lead_days,
CASE WHEN actual_delivery IS NOT NULL AND expected_delivery IS NOT NULL
THEN DATEDIFF(TO_DATE(actual_delivery), TO_DATE(expected_delivery))
ELSE 0 END AS days_late
FROM deduped WHERE rn = 1),
with_supplier AS (SELECT t.*, s.company_name AS supplier_name,
s.reliability_score, s.is_preferred, s.lead_time_days AS supplier_lead_time_days
FROM typed t LEFT JOIN bronze.raw_suppliers s ON t.supplier_id = s.supplier_id)
SELECT *, current_timestamp() AS _loaded_at FROM with_supplier;

```

CELL 9 — silver.silver_returns | PK: return_id

Source: *bronze.raw_returns* · **Key derived cols:** *days_to_restock, is_resellable, reason_group*

```

CREATE OR REPLACE TABLE silver.silver_returns
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY return_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_returns WHERE return_id IS NOT NULL)
SELECT return_id, order_id, sku,
TO_DATE(return_date) AS return_date,
TO_DATE(restock_date) AS restock_date,
CAST(qty_returned AS INT) AS qty_returned,
CAST(refund_amount AS DOUBLE) AS refund_amount,
reason_code, UPPER(TRIM(condition)) AS condition, disposition,
CASE WHEN restock_date IS NOT NULL
THEN DATEDIFF(TO_DATE(restock_date), TO_DATE(return_date))

```

```

ELSE NULL END AS days_to_restock,
UPPER(TRIM(condition)) IN ('NEW', 'LIKE_NEW') AS is_resellable,
CASE WHEN UPPER(reason_code) IN ('DAMAGED','DEFECTIVE') THEN 'Quality'
WHEN UPPER(reason_code) IN ('NOT_AS_DESCRIBED','WRONG_ITEM') THEN 'Accuracy'
WHEN UPPER(reason_code) = 'CHANGED_MIND' THEN 'Customer'
ELSE 'Other' END AS reason_group,
current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;

```

CELL 10 — silver.silver_supplier_performance | PK: perf_id

Source: *bronze.raw_supplier_performance + bronze.raw_suppliers (lookup)* · **Key derived cols:** *composite_score (weighted), score_tier*

```

CREATE OR REPLACE TABLE silver.silver_supplier_performance
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY perf_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_supplier_performance WHERE perf_id IS NOT NULL),
scored AS (SELECT perf_id, supplier_id,
CAST(on_time_rate AS DOUBLE) AS on_time_rate,
CAST(fill_rate AS DOUBLE) AS fill_rate,
CAST(defect_rate AS DOUBLE) AS defect_rate,
CAST(avg_lead_days AS DOUBLE) AS avg_lead_days,
-- composite = 40% on-time + 40% fill + 20% quality
(CAST(on_time_rate AS DOUBLE) * 0.4)
+ (CAST(fill_rate AS DOUBLE) * 0.4)
+ ((1.0 - CAST(defect_rate AS DOUBLE)) * 0.2) AS composite_score
FROM deduped WHERE rn = 1),
tiered AS (SELECT *,
CASE WHEN composite_score >= 0.95 THEN 'A'
WHEN composite_score >= 0.85 THEN 'B'
ELSE 'C' END AS score_tier
FROM scored),
with_supplier AS (SELECT t.*, s.company_name AS supplier_name,
s.is_preferred, s.country
FROM tiered t LEFT JOIN bronze.raw_suppliers s ON t.supplier_id = s.supplier_id)
SELECT *, current_timestamp() AS _loaded_at FROM with_supplier;

```

CELL 11 — silver.silver_employees | PK: employee_id

Source: *bronze.raw_employees + bronze.raw_warehouses (lookup)* · **Key derived cols:** *tenure_days, annual_cost_est*

```

CREATE OR REPLACE TABLE silver.silver_employees
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY employee_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_employees WHERE employee_id IS NOT NULL),
typed AS (SELECT employee_id, first_name, last_name, role, warehouse_id,
TO_DATE(hire_date) AS hire_date,
CAST(hourly_rate AS DOUBLE) AS hourly_rate,
CAST(is_active AS BOOLEAN) AS is_active,
DATEDIFF(CURRENT_DATE(), TO_DATE(hire_date)) AS tenure_days,
CAST(hourly_rate AS DOUBLE) * 2080.0 AS annual_cost_est
FROM deduped WHERE rn = 1),
with_warehouse AS (SELECT t.*, w.name AS warehouse_name, w.region AS warehouse_region
FROM typed t LEFT JOIN bronze.raw_warehouses w ON t.warehouse_id = w.warehouse_id)
SELECT *, current_timestamp() AS _loaded_at FROM with_warehouse;

```

CELL 12 — silver.silver_labor_shifts | PK: shift_id

Source: *bronze.raw_labor_shifts + bronze.raw_employees (lookup)* · **Key derived cols:** *picks_per_hour, units_per_hour, labor_cost*

```

CREATE OR REPLACE TABLE silver.silver_labor_shifts
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY shift_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_labor_shifts WHERE shift_id IS NOT NULL),
typed AS (SELECT shift_id, employee_id, warehouse_id, zone_assigned,
shift_type, TO_DATE(shift_date) AS shift_date,
CAST(hours_worked AS DOUBLE) AS hours_worked,
CAST(picks_completed AS INT) AS picks_completed,
CAST(units_processed AS INT) AS units_processed,
DATE_FORMAT(TO_DATE(shift_date), 'yyyy-MM') AS shift_month,
CASE WHEN CAST(hours_worked AS DOUBLE) > 0
THEN CAST(picks_completed AS DOUBLE) / CAST(hours_worked AS DOUBLE)
ELSE 0.0 END AS picks_per_hour,
CASE WHEN CAST(hours_worked AS DOUBLE) > 0
THEN CAST(units_processed AS DOUBLE) / CAST(hours_worked AS DOUBLE)
ELSE 0.0 END AS units_per_hour
FROM deduped WHERE rn = 1),
with_employee AS (SELECT t.*, e.first_name, e.last_name, e.role, e.hourly_rate,
t.hours_worked * e.hourly_rate AS labor_cost
FROM typed t LEFT JOIN bronze.raw_employees e ON t.employee_id = e.employee_id)
SELECT *, current_timestamp() AS _loaded_at FROM with_employee;

```

CELL 13 — silver.silver_dock_activity | PK: dock_id

Source: bronze.raw_dock_activity · Key derived cols: activity_date, activity_hour, pallets_per_hour, is_inbound

```

CREATE OR REPLACE TABLE silver.silver_dock_activity
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY dock_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_dock_activity WHERE dock_id IS NOT NULL)
SELECT dock_id, warehouse_id, carrier_id, shipment_id,
UPPER(TRIM(activity_type)) AS activity_type,
TO_TIMESTAMP(arrived_at) AS arrived_at,
TO_TIMESTAMP(departed_at) AS departed_at,
CAST(duration_minutes AS INT) AS duration_minutes,
CAST(num_pallets AS INT) AS num_pallets,
TO_DATE(arrived_at) AS activity_date,
HOUR(TO_TIMESTAMP(arrived_at)) AS activity_hour,
CASE WHEN CAST(duration_minutes AS INT) > 0
THEN CAST(num_pallets AS DOUBLE) / (CAST(duration_minutes AS DOUBLE) / 60.0)
ELSE NULL END AS pallets_per_hour,
UPPER(TRIM(activity_type)) = 'INBOUND_RECEIVE' AS is_inbound,
current_timestamp() AS _loaded_at
FROM deduped WHERE rn = 1;

```

CELL 14 — silver.silver_locations | PK: location_id

Source: bronze.raw_locations · Key derived cols: clean typing and dedup only

```

CREATE OR REPLACE TABLE silver.silver_locations
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')
AS
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (
PARTITION BY location_id ORDER BY _loaded_at DESC) AS rn
FROM bronze.raw_locations WHERE location_id IS NOT NULL)
SELECT location_id, warehouse_id, zone, aisle, bay, level, position,
UPPER(TRIM(location_type)) AS location_type,
CAST(max_weight_kg AS DOUBLE) AS max_weight_kg,
CAST(max_volume_cuft AS DOUBLE) AS max_volume_cuft,
CAST(is_active AS BOOLEAN) AS is_active,

```

```
current_timestamp() AS _loaded_at  
FROM deduped WHERE rn = 1;
```

CELL 15 — silver.silver_date_dim | PK: date_key

Source: bronze.raw_date_dim · Key derived cols: month_label for consistent yyyy-MM labeling in reports

```
CREATE OR REPLACE TABLE silver.silver_date_dim  
USING DELTA TBLPROPERTIES ('delta.autoOptimize.optimizeWrite' = 'true')  
AS  
WITH deduped AS (SELECT *, ROW_NUMBER() OVER (  
PARTITION BY date_key ORDER BY _loaded_at DESC) AS rn  
FROM bronze.raw_date_dim WHERE date_key IS NOT NULL)  
SELECT CAST(date_key AS INT) AS date_key,  
TO_DATE(full_date) AS full_date,  
CAST(day_of_week AS INT) AS day_of_week, day_name,  
CAST(week_num AS INT) AS week_num, CAST(month_num AS INT) AS month_num,  
month_name, CAST(quarter AS INT) AS quarter, CAST(year AS INT) AS year,  
fiscal_period,  
CAST(is_weekend AS BOOLEAN) AS is_weekend,  
CAST(is_holiday AS BOOLEAN) AS is_holiday,  
CONCAT(CAST(CAST(year AS INT) AS STRING), '-',  
LPAD(CAST(CAST(month_num AS INT) AS STRING), 2, '0')) AS month_label,  
current_timestamp() AS _loaded_at  
FROM deduped WHERE rn = 1;
```

Cell 16 — Full Silver Validation

Row count + PK null + duplicate check across all 15 Silver tables. All status values must read ✓ OK before running Part 3 (Gold).

```
WITH audit AS (  
  SELECT 'silver_orders' AS tbl, 'order_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT order_id) AS dupes  
  FROM silver.silver_orders  
  UNION ALL SELECT 'silver_order_lines' AS tbl, 'line_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN line_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT line_id) AS dupes  
  FROM silver.silver_order_lines  
  UNION ALL SELECT 'silver_customers' AS tbl, 'customer_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT customer_id) AS dupes  
  FROM silver.silver_customers  
  UNION ALL SELECT 'silver_products' AS tbl, 'sku' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN sku IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT sku) AS dupes  
  FROM silver.silver_products  
  UNION ALL SELECT 'silver_inventory' AS tbl, 'inventory_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN inventory_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT inventory_id) AS dupes  
  FROM silver.silver_inventory  
  UNION ALL SELECT 'silver_inventory_transactions' AS tbl, 'txn_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN txn_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT txn_id) AS dupes  
  FROM silver.silver_inventory_transactions  
  UNION ALL SELECT 'silver_shipments' AS tbl, 'shipment_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN shipment_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT shipment_id) AS dupes  
  FROM silver.silver_shipments  
  UNION ALL SELECT 'silver_purchase_orders' AS tbl, 'po_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN po_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT po_id) AS dupes  
  FROM silver.silver_purchase_orders  
  UNION ALL SELECT 'silver_returns' AS tbl, 'return_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN return_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT return_id) AS dupes  
  FROM silver.silver_returns  
  UNION ALL SELECT 'silver_supplier_performance' AS tbl, 'perf_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN perf_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT perf_id) AS dupes  
  FROM silver.silver_supplier_performance  
  UNION ALL SELECT 'silver_employees' AS tbl, 'employee_id' AS pk,  
  COUNT(*) AS rows,  
  SUM(CASE WHEN employee_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,  
  COUNT(*) - COUNT(DISTINCT employee_id) AS dupes  
  FROM silver.silver_employees
```



```

UNION ALL SELECT 'silver_labor_shifts' AS tbl, 'shift_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN shift_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,
COUNT(*) - COUNT(DISTINCT shift_id) AS dupes
FROM silver.silver_labor_shifts
UNION ALL SELECT 'silver_dock_activity' AS tbl, 'dock_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN dock_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,
COUNT(*) - COUNT(DISTINCT dock_id) AS dupes
FROM silver.silver_dock_activity
UNION ALL SELECT 'silver_locations' AS tbl, 'location_id' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN location_id IS NULL THEN 1 ELSE 0 END) AS pk_nulls,
COUNT(*) - COUNT(DISTINCT location_id) AS dupes
FROM silver.silver_locations
UNION ALL SELECT 'silver_date_dim' AS tbl, 'date_key' AS pk,
COUNT(*) AS rows,
SUM(CASE WHEN date_key IS NULL THEN 1 ELSE 0 END) AS pk_nulls,
COUNT(*) - COUNT(DISTINCT date_key) AS dupes
FROM silver.silver_date_dim
)
SELECT tbl, pk AS primary_key, rows, pk_nulls, dupes,
CASE WHEN pk_nulls = 0 AND dupes = 0 THEN '✓ OK' ELSE 'X FIX' END AS status
FROM audit ORDER BY tbl;
-- All 15 rows must show status = '✓ OK' before running Part 3.

```

Notes

Running order: Cells 1–15 can run in the order shown. Cells with no inter-Silver dependencies (3, 4, 9, 13, 14, 15) are safe to run concurrently. Cells that join Bronze reference tables (2, 5, 7, 8, 10, 11, 12) depend only on Bronze being complete.

Deduplication strategy: ROW_NUMBER() OVER (PARTITION BY pk ORDER BY _loaded_at DESC) keeps the most recently loaded record. WHERE rn = 1 filters to that row. Equivalent to dropDuplicates() in PySpark but expressed entirely in SQL.

Reference joins from Bronze (not Silver): Silver tables join Bronze reference data (raw_products, raw_carriers, raw_suppliers, raw_warehouses, raw_employees) rather than their Silver equivalents to avoid Silver-to-Silver ordering dependencies.

PySpark usage in Part 2: ZERO LINES.